tarefa_scipy

October 4, 2026

1. determinante de uma matriz

[1]:
```
import numpy as np
from scipy import linalg as la

A = np.array([
    [4, 2],
    [3, 5]
])

print(la.det(A))
```

```
14.0
```

2. resolver um sistema linear

[2]:
```
A = np.array([
    [2, 1],
    [1, 3]
])

b = np.array([5, 6])

x = la.solve(A, b)

print(x)
```

```
[1.8 1.4]
```

3. autovalores

[4]:
```
A = np.array([
    [4, 1],
    [2, 3]
])

avalores, avetores = la.eig(A)

print(f'autovalores de A: {avalores}')
print(f'autovetores de A: {avetores}')
```

```
autovalores de A: [5.+0.j 2.+0.j]
autovetores de A: [[ 0.70710678 -0.4472136 ]
 [ 0.70710678  0.89442719]]
```

4. integração numérica

[5]:
```
from scipy.integrate import quad

f = lambda x: x**3

I, erro = quad(f, 0, 2)

print(I)
print(erro)
```

```
4.0
4.440892098500626e-14
```

5. cálculo de raízes de funções não lineares

[7]:
```
from scipy.optimize import root_scalar

f = lambda x: x**2 - 7

resultado = root_scalar(f, bracket=[2, 3])

print(resultado)
```

```
      converged: True
           flag: converged
 function_calls: 8
     iterations: 7
           root: 2.64575131106459
         method: brentq
```

6. minimização

[81]:
```
from scipy.optimize import minimize

f = lambda x: x**2 - 6*x + 11

resultado = minimize(f, x0=2)

print(f'o mínimo ocorre em x={resultado.x}')
print(f'o valor mínimo é {resultado.fun}')
```

```
o mínimo ocorre em x=[3.00000001]
o valor mínimo é 2.0000000000000018
```

7. interpolação

[12]:
```
from scipy.interpolate import interp1d

pontos = [(0,0), (1,1), (2,4), (3,9)]
x, y = zip(*pontos)

f = interp1d(x, y)

print(f(1.5))
```

```
2.5
```

8. distribuição normal

[13]:
```
from scipy.stats import norm

print(norm.cdf(1.96))
```

```
0.9750021048517795
```

9. função gama

[17]:
```
from scipy.special import gamma

print(gamma(6))
```

```
120.0
```

10. distância euclidiana

[18]:
```
from scipy.spatial.distance import euclidean

p = [1, 2, 3]
q = [4, 6, 3]

d = euclidean(p, q)

print(d)
```

```
5.0
```

11. criar uma matriz esparsa

[21]:
```
from scipy.sparse import coo_array

valores = [1, 2, 3, 4]
linhas = [0, 1, 3, 3]
colunas = [0, 2, 0, 3]

A = coo_array((valores, (linhas, colunas)), shape=(4, 4))
print(A)
```

```
<COOrdinate sparse array of dtype 'int64'
        with 4 stored elements and shape (4, 4)>
  Coords        Values
  (0, 0)        1
  (1, 2)        2
  (3, 0)        3
  (3, 3)        4
```

12. sistema linear esparso

[23]:
```
from scipy.sparse import csr_array
from scipy.sparse.linalg import spsolve

A = csr_array([
    [4, 1, 0],
    [1, 4, 1],
    [0, 1, 4]
])

b = np.array([6, 6, 6])

x = spsolve(A, b)

print(x)
```

```
[1.28571429 0.85714286 1.28571429]
```

13. transformada de Fourier

[24]:
```
from scipy.fft import fft

x = np.array([1, 1, 1, 1])

X = fft(x)

print(X)
```

```
[4.-0.j 0.+0.j 0.-0.j 0.-0.j]
```

14. resolver uma equação diferencial ordinária

[45]:
```
from scipy.integrate import solve_ivp

def f(t, y):
    return -y

t_span = [0, 5]
y0 = [1]

sol = solve_ivp(
```

```
    f,
    t_span,
    y0
)

print(sol)
print(f'y(5) = {sol.y[0, -1]}')
print(f'erro: {np.abs(sol.y[0, -1] - np.exp(-5))}')
```

```
  message: The solver successfully reached the end of the integration interval.
  success: True
   status: 0
        t: [ 0.000e+00  1.000e-01  1.032e+00  1.908e+00  2.787e+00
             3.667e+00  4.549e+00  5.000e+00]
        y: [[ 1.000e+00  9.048e-01  3.566e-01  1.486e-01  6.170e-02
              2.560e-02  1.061e-02  6.756e-03]]
      sol: None
 t_events: None
 y_events: None
     nfev: 44
     njev: 0
      nlu: 0
y(5) = 0.0067560069234891985
erro: 1.805992440373147e-05
```

15. ajuste de uma curva

[48]:
```
from scipy.optimize import curve_fit

x = np.array([0, 1, 2, 3, 4])
y = np.array([1, 3, 5, 7, 9])

def r(x, a, b):
    return a*x + b

popt, pcov = curve_fit(r, x, y)

print(f'a = {popt[0]}\nb = {popt[1]}')
```

```
a = 1.9999999999999998
b = 1.0000000000000002
```

16. comparação NumPy x SciPy

[ ]:
```
A = np.array([
    [2, 1],
    [1, 2]
])
```

```
det_np = np.linalg.det(A)
det_sp = la.det(A)

print(f'numpy: {det_np}')
print(f'scipy: {det_sp}')

print(f'erro numpy: {np.abs(3-det_np)}')''
```

```
numpy: 2.9999999999999996
scipy: 3.0
erro numpy: 4.440892098500626e-16
```

veja que o determinante computado pela NumPy não é exatamente 3, mas possui erro de acordo com o épsilon de máquina $\epsilon \approx 2.22 \times 10^{-16}$ em precisão dupla.

[84]:
```
import sys
print(sys.float_info)
```

```
sys.float_info(max=1.7976931348623157e+308, max_exp=1024, max_10_exp=308,
min=2.2250738585072014e-308, min_exp=-1021, min_10_exp=-307, dig=15,
mant_dig=53, epsilon=2.220446049250313e-16, radix=2, rounds=1)
```

17. decomposição QR

[71]:
```
from scipy.linalg import qr, solve_triangular

A = np.array([
    [1, 1],
    [1, 0],
    [0, 1]
])

b = np.array([2, 3, 4])

Q, R = qr(A, mode='economic')
x = solve_triangular(R, Q.T @ b)

print(f'solução mínimos quadrados: {x}')
```

```
solução mínimos quadrados: [1.33333333 2.33333333]
```

18. SVD

[80]:
```
from scipy.linalg import svd

A = np.array([
    [3, 0],
    [0, 2]
])
```

```
U, S, VT = svd(A)

print(U @ np.diag(S) @ VT)
```

```
[[3. 0.]
 [0. 2.]]
```